Morpho VaultV2 configs FeeWrapper Security Review

Auditors

Alireza Arjmand, Lead Security Researcher

Saw-mon and Natalie, Lead Security Researcher

Om Parikh, Security Researcher

Report prepared by: Lucas Goiriz

August 11, 2026

Contents

1 About Spearbit

Spearbit is a decentralized network of expert security engineers offering reviews and other security related services to Web3 projects with the goal of creating a stronger ecosystem. Our network has experience on every part of the blockchain technology stack, including but not limited to protocol design, smart contracts and the Solidity compiler. Spearbit brings in untapped security talent by enabling expert freelance auditors seeking flexibility to work on interesting projects together.

Learn more about us at spearbit.com

2 Introduction

Morpho is a trustless and efficient lending primitive with permissionless market creation.

Disclaimer: This security review does not guarantee against a hack. It is a snapshot in time of Morpho VaultV2 configs FeeWrapper according to the specific commit. Any modifications to the code will require a new security review.

3 Risk classification

Severity level	**Impact: High**	**Impact: Medium**	**Impact: Low**
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

3.1 Impact

- High - leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium - global losses <10% or losses to only a subset of users, but still unacceptable.
- Low - losses will be annoying but bearable--applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

3.2 Likelihood

- High - almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium - only conditionally possible or incentivized, but still relatively likely
- Low - requires stars to align, or little-to-no incentive

3.3 Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

4 Executive Summary

Over the course of 2 days in total, Morpho engaged with Spearbit to review the vault-v2 protocol. In this period of time a total of **2** issues were found.

Summary

Project Name	Morpho
Repository	vault-v2
Commit	425f6b1f
Type of Project	DeFi, Vaults
Audit Timeline	Jan 7th to Jan 9th

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	0	0	0
Informational	2	0	2
Total	**2**	**0**	**2**

4.1 Scope

The security review had the following components in scope for vault-v2 on commit hash 425f6b1f:

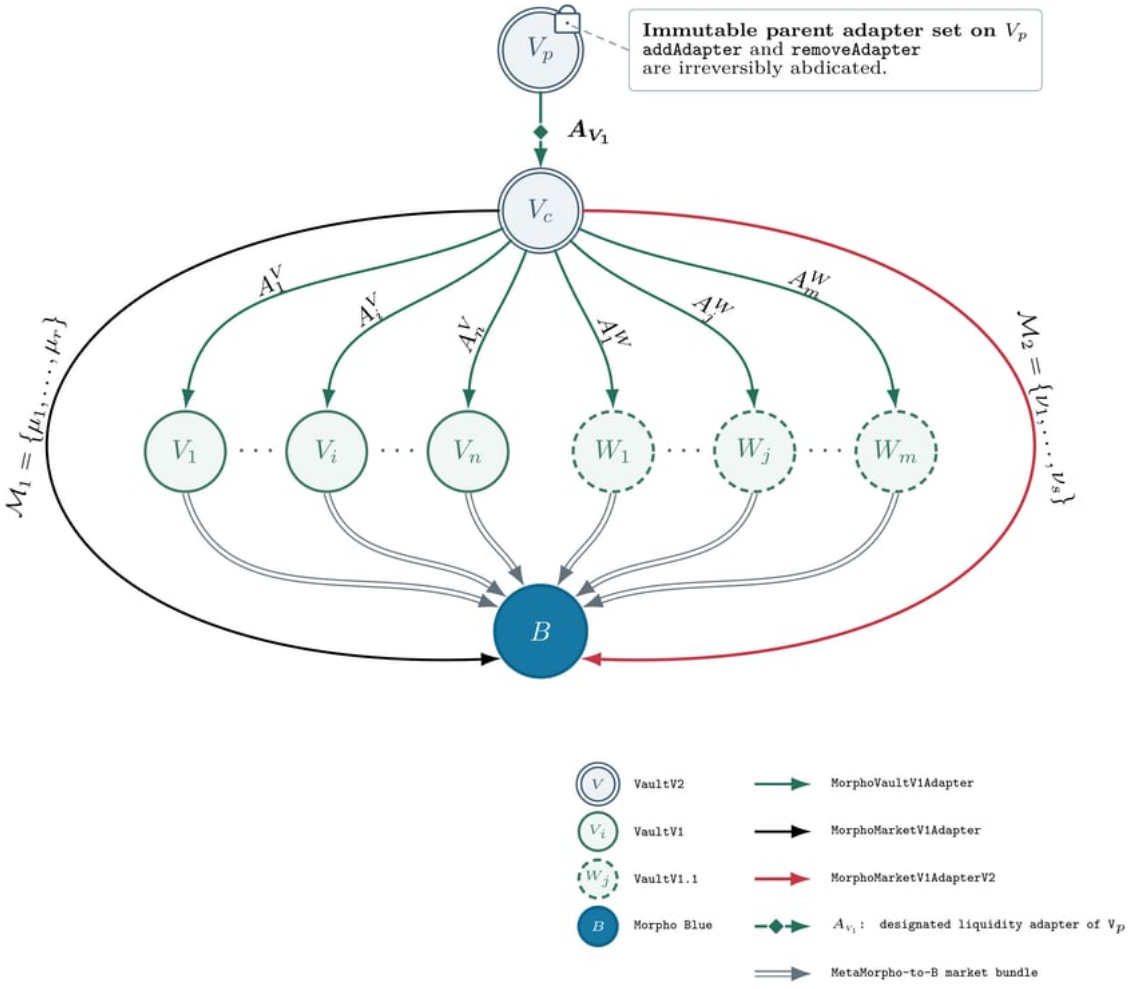
```
src
├── VaultV2.sol
└── adapters
    └── MorphoVaultV1Adapter.so
```

This report evaluates a concrete configuration where a parent `VaultV2` supplies into a single Morpho `VaultV2` using the same adapter, with

- `addAdapter` and...
- `removeAdapter`.

abdicated so the adapter set is frozen. The `liquidityAdapter` is set to `MorphoVaultV1Adapter` and `liquidityData` is empty.

A `VaultV2` (parent vault V_p) with a single `MorphoVaultV1Adapter` the underlying vault (child vault V_c) of this adapter only uses the `MorphoVaultV1Adapter`, the `MorphoMarketV1Adapter` and the `MorphoMarketV1AdapterV2` that themselves point to official **Blue**, `VaultV1`, `VaultV1.1` (**MetaMorpho**) instances.


Immutable parent adapter set on V_p
addAdapter and removeAdapter
are irreversibly abdicated.
V_p
A_{V_1}
V_c
A_1^V
A_i^V
A_n^V
A_1^W
A_j^W
A_m^W
$\mathcal{M}_1 = \{\mu_1, \ldots, \mu_r\}$
$\mathcal{M}_2 = \{\nu_1, \ldots, \nu_s\}$
V_1
V_i
V_n
W_1
W_j
W_m
B
VaultV2
VaultV1
VaultV1.1
Morpho Blue
MorphoVaultV1Adapter
MorphoMarketV1Adapter
MorphoMarketV1AdapterV2
A_{V_1} : designated liquidity adapter of V_p
MetaMorpho-to-B market bundle

5 Findings

5.1 Informational

5.1.1 `ForceDeallocate` Is Required on Child Vault to Exit Parent in Illiquid States

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: In a VaultV2-to-VaultV2 composition, the parent vault (`v_p`) depends on the child vault (`v_c`) for liquidity. If `v_c` is illiquid and there isn't enough balance in `v_p` , a user who wants to withdraw from `v_p` must first invoke `forceDeallocate` on `v_c` to free assets, and then exit via `v_p` . There is no direct "single-step" exit through `v_p` that will successfully unwind an illiquid position in `v_c` . This is expected behavior, but it can be surprising for integrators and users.

Recommendation: Document the required withdrawal flow for illiquid child vaults, users must call `forceDeallocate` on `v_c` to free assets before redeeming or withdrawing from `v_p` .

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.1.2 Allocation mechanism in the presence of allocators

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: When the allocator set is not empty, one could have:

- If a user force deallocates some funds from the child vault, an allocator entity can re-allocate those funds back into the child vault. Thus if force deallocation is present, having at least one allocator is necessary to make sure some funds don't just stay in the idle liquidity.
- If an allocator role disconnects the liquidity adapter, there should be an exit strategy for the user to be able to pull the funds in the child vault back into the parent vault. That is why in this case force deallocation is necessary from the user perspective.

Once can see, the above items kind of create a symmetric requirement from the protocol and user perspectives.

Morpho: Acknowledged.

Spearbit: Acknowledged.